

Plan détaillé — opti_route (version CB)

Application mobile Flutter (iOS + Android) pour l'optimisation de tournées de livraison multi-points, avec OCR d'adresses, contraintes par client, navigation et trafic temps réel. **Version avec carte bancaire** : on utilise Google Maps Platform et ses APIs payantes pour une qualité professionnelle.

✦ **À lire en parallèle de [plan_free.md](#)** : ce document met l'accent sur ce qui change par rapport à la version gratuite. La structure (modèle de données, écrans, jalons) reste très proche.

1. Ce que la version CB débloque vs la version gratuite

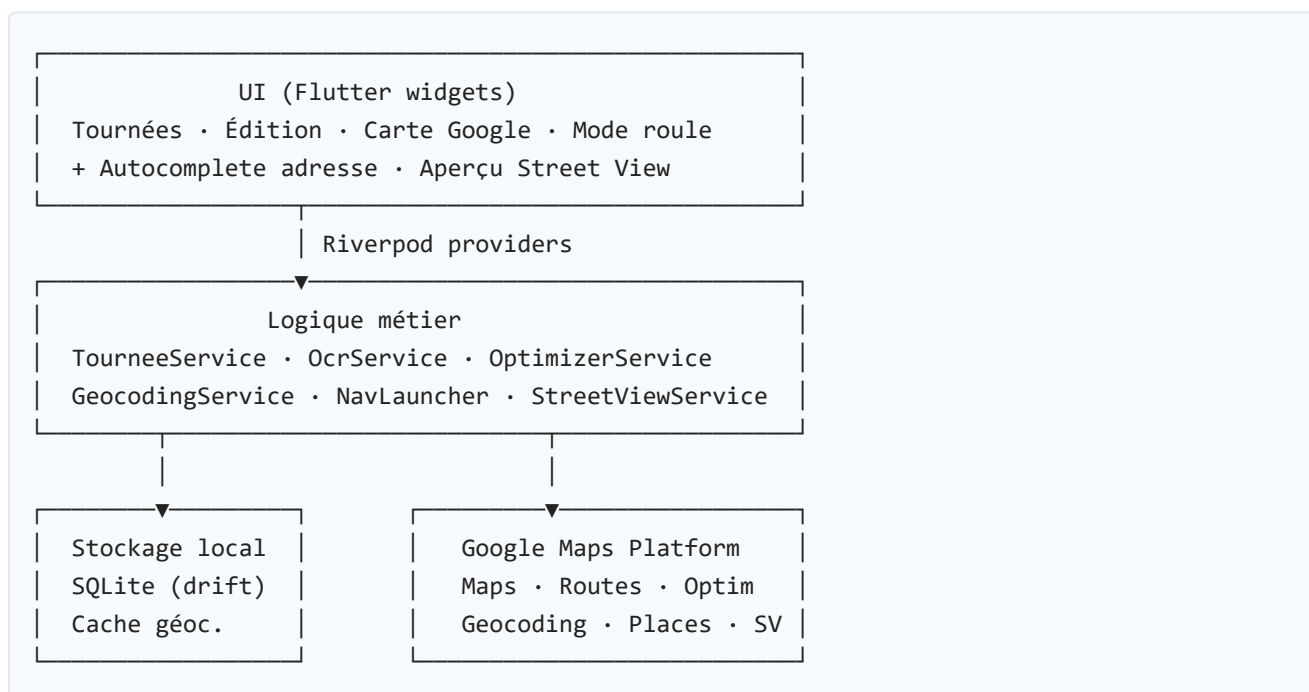
Capacité	Version gratuite	Version CB
Qualité du fond de carte	OpenStreetMap (correct, parfois daté en zones rurales)	Google Maps (référence mondiale, vue satellite, Street View)
Trafic temps réel	TomTom 2500 req/jour	Google Maps Traffic (inclut les données issues de Waze, sans limite mensuelle au-delà du crédit)
Optimisation tournée	OpenRouteService / VROOM (500/jour)	Google Route Optimization API (solveur dédié livraison, contraintes plus riches)
Géocodage	Nominatim (parfois imprécis)	Google Geocoding API (très précis, gère les codes postaux partiels et les fautes)
Saisie d'adresse	Champ texte simple	Places Autocomplete (suggestions instantanées, tolérant aux fautes)
Visualisation arrêt	Aucune	Street View Static (voir la façade avant d'arriver)
OCR	ML Kit (gratuit, identique)	ML Kit (gratuit, identique)
CarPlay / Android Auto	Lancement nav externe	Toujours code natif requis, mais données Google disponibles

Ce que la CB ne change PAS : - ✗ **Pas d'API Waze séparée**. Waze n'a aucune API publique, peu importe le budget. Cela dit, le trafic Google Maps **intègre** les données collectées par Waze depuis l'acquisition en 2013 — donc en pratique, tu as les données Waze, juste pas le branding. - ✗ **CarPlay / Android Auto ne sont pas magiques**. La Google Navigation SDK qui supporte CarPlay nativement est réservée aux partenaires entreprise (pas dispo en self-service). Tu peux utiliser Google Maps Platform pour tout sauf la nav in-app sur CarPlay — pour ça, soit tu lances l'app Google Maps externe, soit tu codes la nav toi-même côté natif Swift/Kotlin.

2. Stack technique

Couche	Outil	Pourquoi
Framework	Flutter 3.x (Dart)	Idem version free. Un seul code iOS + Android.
Carte	google_maps_flutter	SDK officiel Google, perf native, vue satellite, Street View.
Optimisation tournée	Google Route Optimization API	Solveur Google de classe mondiale. Gère multi-véhicules, fenêtres horaires souples, capacités multi-dimensions, compétences.
Itinéraires & temps	Google Routes API (avec <code>TRAFFIC_AWARE_OPTIMAL</code>)	Trafic temps réel inclut les remontées Waze.
Géocodage	Google Geocoding API + Places Autocomplete	Précision excellente. Autocomplete = saisie 5× plus rapide.
Trafic temps réel	Inclus dans Routes API	Pas d'API séparée nécessaire.
Visualisation	Street View Static API	Aperçu de la façade pour repérer la sonnette/code.
OCR	google_mlkit_text_recognition	Identique à la version free, gratuit et hors ligne.
Base de données locale	drift (SQLite)	Identique à la version free.
Gestion d'état	Riverpod	Identique à la version free.
Lancement nav externe	url_launcher + URL schemes	Idem free, pour Phase 3 si pas de SDK Navigation.

3. Architecture (vue d'ensemble)



Principe d'économie : on **cache localement** tout ce qui peut l'être (géocodages, tracés d'itinéraires, vignettes Street View) pour éviter de payer plusieurs fois la même requête. C'est le facteur n°1 pour garder la facture sous les \$200/mois de crédit gratuit.

4. Modèle de données

Identique au plan gratuit, avec quelques ajouts mineurs :

Table stops — colonnes ajoutées

Colonne	Type	Description
place_id_google	text?	ID Google Places, utile pour requêtes futures sans re-géocoder
street_view_url_cache	text?	URL Street View générée (image cachée localement)
precision_geocodage	text	ROOFTOP / RANGE_INTERPOLATED / GEOMETRIC_CENTER / APPROXIMATE

Table cache_api

Pour minimiser les coûts, on cache localement les réponses API. | Colonne | Type | Description | |---|---|---|
| c1e | text (PK) | Hash de la requête (ex: geocode:123 rue Foo, Paris) | | valeur_json | text | Réponse complète | | expire_1e | datetime | TTL (ex: 30 jours pour géocodage, 5 min pour trafic) |

5. Écrans (différences par rapport à la version gratuite)

Les écrans sont les mêmes que [plan_free.md](#) section 4. Voici uniquement les **améliorations** :

Écran 3a — Saisie manuelle d'un arrêt

- **Places Autocomplete** : à mesure que tu tapes « 12 rue de la R... », des suggestions apparaissent. Tu choisis et le géocodage est instantané + gratuit (inclus dans la session Autocomplete).
- Aperçu **Street View** automatique de l'adresse choisie pour validation visuelle.

Écran 3b — Scanner caméra

- Après extraction OCR, si plusieurs candidats d'adresse détectés, **Places Autocomplete** propose des correspondances de la base Google Maps (plus tolérant aux fautes que Nominatim).

Écran 4 — Vue carte + ordre optimisé

- Bascule **Plan / Satellite / Hybride** (Google Maps SDK).
- Tap long sur un arrêt → vignette **Street View** en bottom sheet.
- Tracé du trajet **suit le trafic actuel** (couleurs Google : vert/jaune/rouge sur les segments).

Écran 5 — Mode « en cours »

- Affichage **ETA dynamique** pour le prochain stop (recalculé toutes les ~60s avec Routes API).
- Si embouteillage détecté : alerte « Suggérer une re-optimisation ? ».
- Vignette Street View du prochain stop pour anticiper l'arrivée.

6. Ordre des tâches (jalons Phase 1)

Identique à [plan_free.md](#) section 5, avec **un setup différent au jalon 1** (compte Google Cloud + facturation + activation des APIs) et **les jalons 5-7 simplifiés** car les APIs Google sont plus fluides à intégrer.

#	Jalon	Estimation débutant	Différence vs free
1	Setup + compte GCP avec facturation activée	1 journée	+ ~30 min compte GCP
2	Base de données + table <code>cache_api</code>	1-2 jours	Léger ajout
3	Liste tournées + créer/éditer	2-3 jours	Identique
4	Saisie manuelle + Places Autocomplete	2-3 jours	Plus facile et plus précis
5	Google Geocoding (avec cache)	1 jour	Plus simple que Nominatim
6	google_maps_flutter intégration	1-2 jours	Plus fluide qu'OSM
7	Route Optimization API	2-3 jours	Plus simple que VROOM
8	OCR + Places Autocomplete pour fallback	4-7 jours	Identique
9	Mode roule + ETA dynamique	2-3 jours	+ ETA temps réel
10	Polish + monitoring quotas GCP	2-3 jours	+ dashboard quotas

Total Phase 1 estimé pour un débutant motivé : 5 à 9 semaines (légèrement plus rapide que le [plan_free](#) grâce à la qualité des APIs Google).

7. Comptes & clés à créer (avec carte de crédit)

Avant de coder le jalon 1 :

1. **Google Cloud Platform** : - Créer un compte sur <https://console.cloud.google.com> - **Activer la facturation** (carte bancaire requise) - Crédit gratuit de 300 \$ valable 90 jours pour les nouveaux comptes - Crédit récurrent de **200 \$/mois** sur Google Maps Platform (couvre la majorité des usages perso)
2. **Activer les APIs nécessaires** dans la console GCP : - Maps SDK for Android - Maps SDK for iOS - Geocoding API - Routes API - Route Optimization API (peut nécessiter une demande manuelle si refusée automatiquement) - Places API - Street View Static API

3. **Créer 2 clés API** : - Une pour Android (restreinte par empreinte SHA-1 + nom de package) - Une pour iOS (restreinte par bundle ID)
4. **Configurer des quotas** dans GCP pour éviter les mauvaises surprises : - Quota max/jour par API (ex: max 1000 géocodages/jour) - **Alertes de facturation** à 50, 100, 150 \$ — à configurer impérativement avant la première ligne de code.

8. Coûts estimés (réalistes, usage solo)

Tarifs Google Maps Platform officiels au moment de la rédaction (vérifier sur <https://mapsplatform.google.com/pricing>). Le **crédit mensuel gratuit est de 200 \$**.

Hypothèses : un livreur, 1 tournée/jour, 50 arrêts/tournée, ~22 jours/mois

Poste	Tarif	Volume mensuel	Coût brut
Maps SDK Mobile (chargements dynamiques)	7 \$ / 1000 après 28k gratuits	~3 000 chargements	0 \$ (sous le seuil)
Geocoding API (avec cache 30j)	5 \$ / 1000	~200 nouveaux géoc.	1 \$
Places Autocomplete (per-session)	17 \$ / 1000 sessions	~100 sessions	1,70 \$
Routes API (avec trafic)	10 \$ / 1000	~1 500 (ETA dynamique)	15 \$
Route Optimization API	~5 \$ / 1000 « shipments »	22 × 50 = 1 100 stops	5,50 \$
Street View Static	7 \$ / 1000	~500 vignettes	3,50 \$
Sous-total brut			~26,70 \$/mois
Crédit mensuel offert			-200 \$
À payer réellement			0 \$/mois

Marges de sécurité

- À volume **double** (2 tournées/jour), tu restes encore largement sous les 200 \$/mois.
- Les surprises arrivent quand un bug fait boucler des appels API. **Toujours mettre des plafonds quotidiens** dans la console GCP.
- En cas d'arrêt brutal du projet : aucune facture si tu désactives la facturation de Maps Platform avant.

Ce qui ferait exploser la facture

- Pas de cache géocodage → chaque appel coûte 5 \$/1000 et c'est rapide.
 - Polling Routes API toutes les 5s en mode roule (au lieu de toutes les 60s) → ×12 le coût du trafic.
 - Affichage Street View dans une liste qui défile avec scroll → des centaines d'appels par minute.
-

9. Risques & points d'attention

Risque	Atténuation
Facture surprise	Alertes facturation + quotas max/jour configurés AVANT le premier déploiement.
Restriction de clé API contournée	Toujours restreindre par SHA-1 (Android) / bundle ID (iOS). Ne jamais committer la clé en clair dans Git.
Refus Route Optimization API	Cette API peut demander une justification d'usage. Prévoir un fallback OpenRouteService (code de la version free) pour les premières semaines.
Dépendance Google	Si Google change ses prix ou ses CGU, tu es captif. Garder une couche d'abstraction <code>OptimizerService</code> qui peut basculer vers ORS.
Internet requis	Toutes les APIs sont en ligne. Sans signal : OCR fonctionne, mais pas géocodage ni optim. Prévoir un mode dégradé.
CarPlay toujours non trivial	La CB ne donne PAS accès à Google Navigation SDK pour CarPlay. Reste Phase 3 + code natif Swift.

10. Setup machine

Identique à [plan_free.md](#) section 9, plus :

- **Carte bancaire valide** pour activer la facturation GCP.
- (Recommandé) **Adresse e-mail dédiée** au projet pour le compte GCP, distincte de ton e-mail principal.
- (Recommandé) Activer la **2FA** sur le compte Google qui détient la facturation — clé API volée = facture potentiellement énorme.

11. Recommandation finale

Ne paie pas tant que tu n'as pas un MVP qui tourne.

Stratégie suggérée : 1. **Commencer par la version free** ([plan_free.md](#)) jusqu'à avoir un MVP utilisable au quotidien (jalons 1-7 minimum). 2. Quand tu connais ton vrai usage (combien d'arrêts, combien de tournées, quelles fonctions tu veux vraiment), **basculer sur la version CB** pour les fonctionnalités qui valent le coût : Places Autocomplete, trafic Google, Street View. 3. Garder la couche `OptimizerService` abstraite : tu peux choisir le fournisseur (ORS ou Google) selon une option dans les paramètres.

Ce parcours t'évite de : - Apprendre Flutter et payer des APIs en même temps (charge cognitive). - Découvrir trop tard que tu n'as pas vraiment besoin de Street View ou de Places. - Dépendre de Google avant d'avoir validé que l'app te sert quotidiennement.

12. Question ouverte

Même question que dans plan_free.md : **as-tu un téléphone Android pour tester sur du vrai matériel ?**
Et : **es-tu d'accord avec la stratégie « free d'abord, CB ensuite » ou tu préfères attaquer directement la version CB ?**